DevOps and CI/CD: Automating Processes for Research Projects

Aibyn Talgatov, group CSE-2505M, student ID 255308
Astana IT University, Faculty of Computer Science and Engineering
Applied Software Development Project. Instructor: Akanova Akerke

1. DevOps: goals, principles and benefits

DevOps is a set of cultural and technical practices that joins software development and operations into one shared responsibility for delivering working software. Its goal is to shorten the path from a change to a running, trustworthy system without giving up stability. Its core principles are automation of repetitive work, small and frequent changes, version control for everything, including environments, fast feedback, and measurement. Forsgren, Humble and Kim [1] link these practices to four delivery metrics: deployment frequency, lead time for changes, time to restore service, and change failure rate. Their central finding is that speed and stability improve together rather than being traded against each other. The main benefits are fewer manual errors, repeatable releases, faster recovery, and less knowledge locked in one person's head.

2. Continuous integration and continuous delivery

Continuous integration (CI) means that every change is merged often and verified automatically by a build and a test suite, so defects surface within minutes of being introduced. Continuous delivery (CD) extends this with a deployment pipeline in which every commit that passes all stages is a release candidate, and the release itself is a routine automated step [2]. Continuous deployment goes further and ships each passing build without human approval. A pipeline orders its stages from cheap to expensive, so that most failures are found early, and it builds a single artifact that moves unchanged through all later stages. The practical result is that releasing stops being a risky event.

3. DevOps tools in research environments

Research software has a problem that commercial software rarely faces: its output is a scientific claim. In a *Nature* survey of 1,576 researchers, 52 % agreed that there is a significant reproducibility crisis, and more than 70 % had failed to reproduce another scientist's experiment [3]. Computational work should be the easy case, yet results often depend on an undocumented laptop environment. Three DevOps ideas answer this directly. *Environment drift.* Code that gives one answer today can give another after a library upgrade. Exact dependency pins and container images record the environment instead of describing it [4], [5].

Data versioning. Results depend on data as much as on code. Wilson et al. recommend keeping raw data unchanged and tracking every step that transforms it [6]. Committed, checksummed snapshots make a result traceable to the precise input that produced it.
Tests of the science. Ordinary unit tests show that code runs, not that the answer is right. Research pipelines can also assert physics and statistics: inversions on synthetic data with a known answer, calibrated uncertainties, and byte-identical reruns.

4. Docker, Jenkins and Kubernetes in scientific and engineering projects

Docker packages an application with its dependencies in an image that runs the same way on any host [4]. Boettiger shows its value for reproducible research: it replaces long installation notes with an executable description of the environment [5]. In high-performance computing centres, where running a root-owned daemon is often unacceptable, Singularity applies the same idea [7].

Jenkins is an open-source automation server that runs pipelines defined in a Jenkinsfile stored with the code. Keeping the pipeline and the server configuration in version control means that the automation itself can be reviewed and rebuilt.

Kubernetes is a container orchestrator derived from Google's experience with Borg and Omega [8]. It keeps a declared state: a set number of replicas, restarts of failed containers, health probes, and rolling updates that can be undone. These features suit research services such as data portals and shared models, and Minikube provides a single-node cluster for learning and testing on a laptop.

This report's practical project, *quakeloc*, combines all three. It is a small seismology service for the northern Tien Shan that detects earthquakes (STA/LTA), picks P-wave onsets (AIC) and locates epicentres with 95 % error ellipses (Geiger's method), validated on open data from the KNET network near Bishkek, acquired with ObsPy [9]. A multi-stage Dockerfile builds the test and runtime images, and the service runs as a non-root user. A Jenkins pipeline, itself configured as code, tests every pushed branch: lint, unit and property tests with a coverage gate, then scientific validation. That stage repeats a seeded detector calibration in two separate containers and requires byte-identical results. A further stage checks the relocated events against limits derived from the first run of a frozen real-data snapshot. The pipeline then proves that the calibration baked into the runtime image is the one it tested. Only the main branch pushes the image to Docker Hub and deploys it to Minikube, where a readiness probe refuses traffic to a pod whose built-in test earthquake is mislocated, and a failed smoke job triggers an automatic rollback. Live data APIs are used only by a separate

refresh job, so an outage at a data centre cannot make the main pipeline fail, and new data enters only through a reviewed pull request.

5. Conclusion

DevOps practices are valuable beyond industry because they turn reproducibility from a promise into something a machine checks on every commit. For a research project the cost is moderate and the return is high: containers fix the environment, pipelines run the checks that people forget, versioned snapshots fix the data, and orchestration keeps services healthy. The limits should also be stated. A green pipeline proves only what its tests assert, and a pipeline cannot replace scientific judgement about whether a model is adequate.

References

[1] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution, 2018.

[2] J. Humble and D. Farley, *Continuous Delivery*. Addison-Wesley, 2010.

[3] M. Baker, “1,500 scientists lift the lid on reproducibility,” *Nature*, vol. 533, pp. 452–454, 2016.

[4] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux J.*, no. 239, 2014.

[5] C. Boettiger, “An introduction to Docker for reproducible research,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 49, no. 1, pp. 71–79, 2015.

[6] G. Wilson et al., “Good enough practices in scientific computing,” *PLoS Comput. Biol.*, vol. 13, no. 6, e1005510, 2017.

[7] G. M. Kurtzer, V. Sochat, and M. W. Bauer, “Singularity: Scientific containers for mobility of compute,” *PLoS ONE*, vol. 12, no. 5, e0177459, 2017.

[8] B. Burns et al., “Borg, Omega, and Kubernetes,” *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[9] L. Krischer et al., “ObsPy: A bridge for seismology into the scientific Python ecosystem,” *Comput. Sci. Discov.*, vol. 8, 014003, 2015.